Chapter 6

SYSTEM IMPLEMENTATION

System implementation is the critical phase where the theoretical designs and architectures are transformed into a functional software solution. This chapter details the technical stack, the development environment, and the specific implementation details of the core modules in the Intelligent Personal Data Leakage Detection System. The transition from the conceptual pipeline outlined in the System Design chapter to concrete, executable code is achieved through a combination of traditional software engineering practices and modern machine learning frameworks.

6.1 Development Environment and Tools

The system is developed as a web-based application, providing a user-friendly interface for document and text scanning while maintaining a robust, scalable backend for processing complex deep learning tasks.

- **Programming Language:** Python 3.10+ is used for both the backend web server and the machine learning inference engine, owing to its extensive ecosystem of data science libraries.
- **Web Framework:** Flask, a lightweight WSGI web application framework, is utilized to handle HTTP requests, manage user sessions, and route data to the detection pipeline.
- **Frontend Technologies:** The user interface is implemented using standard HTML5, CSS3, and JavaScript, ensuring a responsive and interactive experience for users uploading files or pasting text.
- **Deep Learning Frameworks:** PyTorch and the Hugging Face Transformers library are utilized for loading and running the fine-tuned BERT models.
- **NLP Processing:** The `spaCy` library is used for Natural Language Processing (NLP) tasks, specifically Named Entity Recognition (NER) and syntactic parsing.

- **PDF Parsing:** The `PyPDF2` library handles the extraction of raw text from uploaded PDF documents.

6.2 Implementation of Detection Modules

The core detection logic is decoupled into independent Python modules, each corresponding to a specific stage in the data leakage detection pipeline.

6.2.1 Rule-Based Detection

The rule-based module (`rule_based.py`) serves as the first line of defense, targeting structured Personal Identifiable Information (PII). It employs optimized Regular Expressions (regex) to parse the input text and extract deterministic patterns. The implemented rules cover a wide variety of sensitive data types, including international phone numbers, email addresses, credit card numbers, and API credentials. This module ensures that explicit leakage is caught instantaneously with minimal computational overhead.

6.2.2 NLP-Based Contextual Analysis

The NLP module (`nlp_contextual.py`) addresses unstructured text where regex falls short. It utilizes the `spaCy` library to perform Named Entity Recognition. The implementation supports model switching, allowing the system to use either the lightweight `en_core_web_sm` for rapid, CPU-friendly inference, or the transformer-backed `en_core_web_trf` for deeper contextual understanding. This module extracts entities such as Person Names, Organizations, and Locations, and evaluates the surrounding syntactic context to determine if the entity represents a leakage risk.

6.2.3 Deep Learning Classification

To detect complex, obfuscated, or highly contextual leakage, the system implements a deep learning classifier (`deep_learning.py`). This module integrates a fine-tuned BERT (Bidirectional Encoder Representations from Transformers) model via the Hugging Face `transformers` pipeline. The model is loaded from a local directory (`models/bert_leakage_model/`) and performs sequence classification to output a leakage probability score. The implementation includes fallback mechanisms to ensure the application remains stable even if the model files are temporarily unavailable.

6.2.4 Behavioral Anomaly Detection

The anomaly detection module (`anomaly.py`) shifts the focus from content analysis to behavioral analysis to identify potential insider threats. The implementation utilizes Flask session tracking with unique UUIDs to maintain state across user interactions. It logs behavioral metrics, including the time of day, the volume of data submitted, and the historical sensitivity of the user's

scans. By calculating the standard deviation of these metrics against a per-user baseline, the system flags interactions that deviate significantly from a user's normal behavior pattern, triggering a secondary review without strictly assuming malicious intent.

6.3 Risk Scoring and Explainability

Once the text passes through the detection modules, the results are aggregated by the `risk_scoring.py` module. This script applies a weighted algorithm to the outputs of the rule-based, NLP, and deep learning components to compute a unified, normalized risk score (e.g., Low, Medium, High, Critical).

Finally, the `explainability.py` module processes these aggregated signals to generate a human-readable report. This report is returned to the Flask frontend via JSON and rendered dynamically using JavaScript. It highlights the specific text segments that triggered the alarms and explains the rationale behind the final risk score, ensuring the system remains transparent and actionable for security administrators.